

P2723R1

Zero-initialize objects of automatic storage duration

Published Proposal, 2023-01-15

This version:

<http://wg21.link/P2723>

Author:

[JF Bastien](#) (Woven Planet)

Audience:

EWG, SG12, SG22

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Source:

github.com/jfbastien/papers/blob/master/source/P2723r1.bs

Table of Contents

1	Summary
2	Opting out
2.1	<code>[[uninitialized]]</code> attribute
2.2	<code>void</code> keyword
2.3	<code>std::uninitialized</code> library type
2.4	Weird Idiom
2.5	Tooling
3	Security details
3.1	Undefined Behavior
3.2	Compiler Optimizations
3.3	Examples

- 3.4 Security Experts
- 3.5 In the Real World

4 Alternatives

- 4.1 Tooling
- 4.2 Trying harder
- 4.3 `-Werror=uninitialized`
- 4.4 Definitive initialization
- 4.5 Value initialization
- 4.6 Fixing initialization
- 4.7 Do nothing
- 4.8 Other values
- 4.9 Implementation defined
- 4.10 Hybrid approach
- 4.11 Overwrite
- 4.12 Rust `std::uninitialized`
- 4.13 Wobbly bits
- 4.14 New Word of Power
 - 4.14.1 Erroneous Behavior
 - 4.14.2 Other Words

5 Performance

6 Caveats

7 Wording

8 Feedback

References

Informative References

§ 1. Summary

Reading uninitialized values is undefined behavior which leads to well-known security issues [\[CWE-457\]](#), from information leaks to attacker-controlled values being used to control execution,

leading to an exploit. This can occur when:

- The compiler re-uses stack slots, and a value is used uninitialized;
- The compiler re-uses a register, and a value is used uninitialized;
- Stack structs / arrays / unions with padding are copied; and
- A heap-allocated value isn't initialized before being used.

This proposal only addresses the above issues when they occur for automatic storage duration objects (on the stack). The proposal does not tackle objects with dynamic object duration. Objects with static or thread local duration are not problematic because they are already zero-initialized if they cannot be constant initialized, even if they are non-trivial and have a constructor execute after this initialization.

[illegible]

```
char *allocated = malloc(128);    // unchanged
int *new_int = new int;           // unchanged
char *new_arr = new char[128];    // unchanged
}
```

This proposal therefore transforms some runtime undefined behavior into well-defined behavior.

Adopting this change would mitigate or extinguish around 10% of exploits against security-relevant codebases (see numbers from [\[AndroidSec\]](#) and [\[InitAll\]](#) for a representative sample, or [\[KCC\]](#) for a long list).

We propose to zero-initialize all objects of automatic storage duration, making C++ safer by default.

This was implemented as an opt-in compiler flag in 2018 for LLVM [\[LLVMReview\]](#) and MSVC [\[WinKernel\]](#), and 2021 in for GCC [\[GCCReview\]](#). Since then it has been deployed to:

- The OS of every desktop, laptop, and smartphone that you own;
- The web browser you're using to read this paper;
- Many kernel extensions and userspace program in your laptop and smartphone; and
- Likely to your favorite videogame console.

Unless of course you like esoteric devices. You can [look for uses on websites such as GitHub](#).

The above codebases contain many millions of lines of code of C, C++, Objective-C, and Objective-C++.

The performance impact is negligible (less than 0.5% regression) to slightly positive (that is, some code gets *faster* by up to 1%). The code size impact is negligible (smaller than 0.5%). Compile-time regressions are negligible. Were overheads to matter for particular coding patterns, compilers would be able to obviate most of them.

The only significant performance/code regressions are when code has very large automatic storage duration objects. We provide an attribute to opt-out of zero-initialization of objects of automatic storage duration. We then expect that programmer can audit their code for this attribute, and ensure that the unsafe subset of C++ is used in a safe manner.

This change was not possible 30 years ago because optimizations simply were not as good as they are today, and the costs were too high. The costs are now negligible. We provide details in [§ 5 Performance](#).

Overall, this proposal is a targeted fix which stands on its own, but should be considered as part of a wider push for type and resource safe C++ such as in [\[P2687r0\]](#).

§ 2. Opting out

§ 2.1. `[[uninitialized]]` attribute

In deploying this scheme, we've found that some codebases see unacceptable regressions because they allocate large values on the stack and see performance / size regressions. We therefore propose standardizing an attribute which denotes intent, `[[uninitialized]]`. When put on automatic variables, no initialization is performed. The programmer is then responsible for what happens because they are back to C++23 behavior. This makes C++ safe by default for this class of bugs, and provides an escape hatch when the safety is too onerous.

An `[[uninitialized]]` attribute was proposed in [\[P0632r0\]](#).

§ 2.2. `void` keyword

An alternative for the `[[uninitialized]]` attribute is to use a keyword. We can argue about which keyword is right. However, maybe through [\[P0146r1\]](#), we'll eventually agree that `void` is the right name for "This variable is not a variable of honor... no highly esteemed deed is commemorated here... nothing valued is here." This approach meshes well with a definitive initialization approach because we would keep the semantics of "no value was intended on this variable".

§ 2.3. `std::uninitialized` library type

Keywords are distasteful to some people, and some other people would rather have a named type that they can overload on. For these folks, we might instead consider a magical monostate standard library type name `std::uninitialized`. Magical monostate standard library types are distasteful to some people, who will get to argue for a keyword as above. This type could then be used in a variety of interesting manners:

```
int a = std::uninitialized;
int* b = new int(std::uninitialized);
```

```
auto c = std::make_unique(std::uninitialized);
```

§ 2.4. Weird Idiom

An idiom that is sometimes used to denote "uninitialized on purpose" is `int value = value;`. This is confusing and weird, but people like to bring it up. Let's agree to not.

§ 2.5. Tooling

As a quality of implementation tool, and to ease transitions, some implementations might choose to diagnose on large stack values that might benefit from this attribute. We caution implementations to avoid doing so in their front-end, but rather to do this as a post-optimization annotation to reduce noise. This was done in LLVM [\[AutoInitSummary\]](#).

§ 3. Security details

What exactly is the security issues?

§ 3.1. Undefined Behavior

Currently, uninitialized stack variables are Undefined Behavior if they are used (it is fine to copy them around, but not to use them). The typical outcome is that the program reads stale stack or register values, that is whatever previous value happened to be compiled to reside in the same stack slot or register. This leads to bugs. Here are the potential outcomes:

- *Benign*: in the best case the program causes an unexpected result.
- *Exploit*: in the worst case it leads to an exploit. There are three outcomes that can lead to an exploit:
 - *Read primitive*: this exploit could be formed from reading leaked secrets that reside on the stack or in registers, and using this information to exploit another bug. For example, leaking an important address to defeat ASLR.
 - *Write primitive*: an uninitialized value is used to perform a write to an arbitrary address. Such a write can be transformed into an execute primitive. For example a stack slot is used on a particular execution path to determine the address of a write. The attack can control the previous

stack slot's content, and therefore control where the write occurs. Similarly, an attacker could control which value is written, but not where it is written.

- *Execute primitive*: an uninitialized value is used to call a function.

§ 3.2. Compiler Optimizations

Compilers are also allowed to assume that programmers don't do silly things like this, and optimize assuming code that would read uninitialized state is never reached. Programmers rarely find it hilarious when compilers do this type of thing, but exploit writers might rejoice.

§ 3.3. Examples

Here are a few examples:

```
int get_hw_address(struct device *dev, struct user *usr) {
    unsigned char addr[MAX_ADDR_LEN];          // leak this memory 😞

    if (!dev->has_address)
        return -EOPNOTSUPP;

    dev->get_hw_address(addr);                  // if this doesn't fill add
    return copy_out(usr, addr, sizeof(addr)); // copies all
}
```

Example from [\[LinuxInfoleak\]](#). What can this leak? ASLR information, or anything from another process. ASLR leak can enable Return Oriented Programming (ROP) or make an arbitrary write effective (e.g., figure out where high-value data structures are). Even padding has been shown to leak secrets.

```
int queue_manage() {
    struct async_request *backlog;              // uninitialized

    if (engine->state == IDLE)                  // usually true
        backlog = get_backlog(&engine->queue);
}
```

```
if (backlog)
    backlog->complete(backlog, -EINPROGRESS); // oops 🔥🔥🔥

return 0;
}
```

Example from [\[LinuxUse\]](#). The attacker can control the value of `backlog` by making a previous function call, and ensuring the same stack slot gets reused.

§ 3.4. Security Experts

Security-minded folks think that initializing stack values is a good idea. For example, the Microsoft Windows security team [\[WinKernel\]](#) say:

Between 2017 and mid 2018, this feature would have killed 49 MSRC cases that involved uninitialized struct data leaking across a trust boundary. It would have also mitigated a number of bugs involving uninitialized struct data being used directly.

To date, we are seeing noise level performance regressions caused by this change. We accomplished this by improving the compiler’s ability to kill redundant stores. While everything is initialized at declaration, most of these initializations can be proven redundant and eliminated.

They use pure zero initialization, and claim to have taken the overheads down to within noise. They provide more details in [\[CppConMSRC\]](#) and [\[InitAll\]](#). Don’t just trust Microsoft’s Windows security team though, here’s one of the upstream Linux Kernel security developer asking for this [\[CLessDangerous\]](#), and Linus agreeing [\[Linus\]](#). [\[LinuxExploits\]](#) is an overview of a real-world execution control exploit using an uninitialized stack variable on Linux. It’s been proposed for GCC [\[init-local-vars\]](#) and LLVM [\[LocalInit\]](#) before.

§ 3.5. In the Real World

In Chrome we find 1100+ security bugs for “use of uninitialized value” [\[ChromeUninit\]](#).

12% of all exploitable bugs in Android are of this kind according to [\[AndroidSec\]](#).

Kostya Serebryany has a long list of issues caused by usage of uninitialized stack variables [\[KCC\]](#).

§ 4. Alternatives

When this is discussed, the discussion often goes to a few places.

§ 4.1. Tooling

First, people will suggest using tools such as memory sanitizer or valgrind. Yes they should, but doing so:

- Requires compiling everything in the process with memory sanitizer.
- Can't deploy in production.
- Only find that is executed in testing.
- 3× slower and memory-hungry.

§ 4.2. Trying harder

The next suggestion is usually couldn't you just test / fuzz / code-review better or just write perfect code? However:

- We are (well, except the perfect code part).
- It's not sufficient, as evidenced by the exploits.
- Attackers find what programmers don't think of, and what fuzzers don't hit.

§ 4.3. `-Werror=uninitialized`

The annoyed suggester then says "couldn't you just use `-Werror=uninitialized` and fix everything it complains about?" This is similar to the [\[CoreGuidelines\]](#) recommendation. You are beginning to expect shortcoming, in this case:

- Too much code to change.
- Current `-Wuninitialized` is low quality, has false positives and false negatives. We could get better analysis if we had a compiler-frontend-based IR, but it still wouldn't be noise-free.
- Similarly, static analysis isn't smart enough.
- The value chosen to initialize doesn't necessarily make sense, meaning that code might move from unsafe to merely incorrect.

- Code doesn't express intent anymore, which leads readers to assume that the value is sensible when it might have simply been there to address the diagnostic.
- It prevents checkers (static analysis, sanitizers) from diagnosing code because it was given semantics which aren't intended.

§ 4.4. Definitive initialization

Couldn't you just use definitive initialization in the language? For example as explained in [\[CppConDefinite\]](#) and [\[CppConComplexity\]](#). This one is interesting. Languages such as C# and Swift allow uninitialized values, but only if the compiler can prove that they're not used uninitialized (that is, within some set of rules), otherwise they must be initialized. It's similar to enforcing `-Wuninitialized`, depending on what is standardized. We then have a choice: standardize a simple analysis (such as "must be initialized within the block or before it escapes") and risk needless initializations (with mitigations explained in the prior references), or standardize a more complex analysis. We would likely want to deploy a low-code-churn change, therefore a high-quality analysis. This would require that the committee standardize the analysis, lest compilers diverge. But a high-quality analysis tends to be complex, and change as it improves, which is not sensible to standardize. This would leave us standardizing a high-code-churn analysis with much more limited visibility. Further with definitive initialization, it's then unclear whether `int derp = 0;` lends meaning to `0`, or whether it's just there to shut that warning up.

§ 4.5. Value initialization

One alternative that was suggested instead of zero-initialization is to value-initialize. This would invoke constructors in some cases and default initialize in others, and therefore be a no-go as a significant behavior change (with constructors being called) or a continuation of the uninitialized status quo (when default initialization is performed).

Something along these lines [is implemented in the Circle compiler](#) under `#pragma feature default_value_initialization`.

That said, some form of "trivial enough" initialization might be an acceptable approach to increase correctness (not merely safety) for types for which zero isn't a valid value.

§ 4.6. Fixing initialization

Alisdair Meredith makes such a case. The sources of indeterminate values can be eliminated as a group if we consider that the largest source of such values is default initialization of scalar types, classes with a trivial default constructor, aggregates containing either of both of the previous categories, and data members of such type that are not initialized by a class's constructor.

If we were to eliminate default initialization entirely, and have only value initialization, then this category of issue goes away entirely, and the language simpler and easier to learn. The behavior is not only well defined, but intuitive. We lose corner cases that would lead to folks falling into vexing parses, so explain that less often to students. The term "default constructed" is no longer ambiguous, ending many pedantic conversations. Once value initialization is the only semantic, the term "default Initialization" can be repurposed to mean exactly that.

Once values are always determinate (even if unspecified) then a whole category of reference abuse, notably through function calls, goes away. We do not lose the undefined behavior of a forced bad cast, or an expired object though.

This approach could simplify the language as a whole. Indeed, initialization in C++ is already quite complex. However, it would initialize heap values which, as discussed in this paper, would have unacceptable performance impact.

§ 4.7. Do nothing

We could instead keep the out-of-band option that is already implemented in all major compilers. That out-of-band option is opt-in and keeps the status quo where "C++ is unsafe by default". Further, many compiler implementors are dissatisfied by this option because they believe it's effectively a language fork that users of the out-of-band option will come to rely on (this strategy is mitigated in MSVC by using a non-zero pattern in debug builds according to [\[InitAll\]](#)). These developers would rather see the language standardize the option officially (i.e., standardize existing practice).

§ 4.8. Other values

Rather than zero-initializing, we could use another value. For example, the author implemented a mode where the fundamental types dictate the value used:

- Integral → repeated 0xAA
- Pointers → repeated 0xAA

- Floating-point → repeated 0xFF

This is advantageous in 64-bit platforms because all pointer uses will trap with a recognizable value, though most modern platforms will also trap for large offsets from the null pointer. Floating-point values are NaNs with payload, which propagate on floating-point operations and are therefore easy to root-cause. It is problematic for some integer values that represent sizes in a bounds check, because such a number is effectively larger than any bounds and would potentially permit out-of-bounds accesses that a zero initialization would not.

Unfortunately, this scheme optimizes much more poorly and end up being a roughly 1.5% size and performance regression on some workloads according to [\[GoogleNumbers\]](#) and others.

Using repeated bytes for initialization is critical to effective code generation because instruction sets can materialize repeated byte patterns much more efficiently than any larger value. Any approach other than the above is bound to have a worst performance impact. That is why choosing implementation-defined values or runtime random values isn't a useful approach.

There have been discussions of other more complex schemes (such as using pseudo-random values), all of which perform even worst.

§ 4.9. Implementation defined

Another alternative is to guarantee that accessing uninitialized values is implementation-defined behavior, where implementations *must* choose to either, on a per-value basis:

- zero-initialize; or
- trap (we can argue separately as to whether "trap" is immediate termination, Itanium [\[NaT\]](#), or `std::terminate`, or [ring a bell](#), or something else).

Implementations would then do their best to trap on uninitialized value access, but when they can't prove that an access is uninitialized through however complex optimization scheme, they would revert to zero-initialization. This is an interesting approach which Richard Smith has prototyped in [\[Trap\]](#). However, the author's and others' experience is that such a change is rejected by teams because it cannot be deployed to codebases that seem to work just fine. The Committee might decide to go with this approach, and see whether the default in various implementations is "always zero" or "sometimes trap".

The committee might otherwise choose to leave the object representation as implementation-defined (hence definite), but restrict the valid implementation strategies so that the value representation remains indeterminate. This would disallow trapping in a conforming implementation, but would still prevent

users from relying on a particular value. An implementation could nonetheless elect to trap (in a non-conforming mode), which is similar to what UBSan does on some behaviors (for example, some signed overflows can trap, which is non-conforming).

§ 4.10. Hybrid approach

We could also standardize a hybrid approach that mixes some of the above, creating implementation-defined alternatives which are effectively different build modes that the standard allows. This is what [\[CarbonInit\]](#) does.

§ 4.11. Overwrite

A different proposal can be found in [\[D2488R0\]](#) which proposes attribute `[[for_overwrite]]`. This approach distinguishes itself by having the developer annotate values which are expected to never be read. It does not address the problem at hand in a comprehensive manner.

§ 4.12. Rust `std::uninitialized`

The author has received feedback requesting an approach similar to Rust's `std::uninitialized`. This is a bad idea [as explained by Rust's documentation of the deprecated feature](#):

The reason for deprecation is that the function basically cannot be used correctly: it has the same effect as `MaybeUninit::uninit().assume_init()`. As the `assume_init` documentation explains, the Rust compiler assumes that values are properly initialized.

Truly uninitialized memory like what gets returned here is special in that the compiler knows that it does not have a fixed value. This makes it undefined behavior to have uninitialized data in a variable even if that variable has an integer type.

Therefore, it is immediate undefined behavior to call this function on nearly all types, including integer types and arrays of integer types, and even if the result is unused.

§ 4.13. Wobbly bits

The WG14 C Standards Committee has had extensive discussions about "wobbly values" and "wobbly bits", specifically around [\[DR451\]](#) and [\[N1793\]](#), summarized in [\[Seacord\]](#).

The C Standards Committee has not reached a conclusion for C23, and wobbly bits continue to wobble indeterminately.

§ 4.14. New Word of Power

The C++ Standards Committee isn't stuck with "undefined behavior", "implementation defined behavior", "unspecified behavior", "ill-formed", and "ill-formed, no diagnostic required". Indeed, the Standards Committee made up these words and their meaning, and can make up new words with new and exciting meaning.

§ 4.14.1. Erroneous Behavior

An interesting proposal from the most wonderful Thomas Köppe is to will into being the concept of "erroneous behavior": well-defined behavior which permits the implementation to emit a diagnostic.

Under "erroneous behavior", diagnostics are not tied to the erroneous evaluation, but can happen at any later point, or never.

"Erroneous behavior" acknowledges that real programs contain errors. Erroneous behavior is always a programming error, but it is not always diagnosable by every desirable implementation. An erroneous result is well-defined, but never intentional.

A program execution that does not have erroneous behavior is called "error-free".

An implementation will surely offer a setting that asserts that a program is error-free, which turns any erroneous into undefined behavior and restores the status quo ante. Similarly, a (different) implementation may be able to diagnose all erroneous behavior, but at a cost.

A design question arises: we might want the rule "if a diagnostic is emitted, the behavior is undefined", allowing arbitrary things to happen if an implementation chooses to diagnose an erroneous behavior (e.g., terminate, log-and-continue, etc.). Although it would be nice to bound that somewhat to, say, "either not return, or continue with the specified effects and values".

With this new tool available, we can respecify a previously undefined behavior to a well-defined, erroneous behavior. For example:

A default-initialized automatic variable has an implementation-defined object representation. Performing `gl2pr` on that variable erroneously results in the value implied by that object representation. Note: the compiler must produce that specific, implementation-defined value, because that's the value all further uses see until a diagnostic is raised (if ever). Because the diagnostic may be

raised in a different translation unit at a later point, up to which the behavior remains well-defined, the compiler can't eagerly assume that the value won't be used.

This approach differs from attempts at defining "bounded undefined behavior", by approaching the problem space from the other direction. Instead of modifying and restricting the consequences of undefined behavior, we instead relax the requirements on well-defined behavior, and therefore make well-defined behavior an acceptable choice for certain (otherwise) unacceptable coding patterns. This avoids touching undefined behavior altogether, and instead allows us to selectively strengthen the specification of certain operations where we deem the risk from erroneous use to be high enough to warrant the performance cost. I think it is valuable to call out in the Standard that real programs contain errors, because it is hard otherwise to discuss what we are trying to achieve.

§ 4.14.2. Other Words

The author is certain that we could invent many other fun words, with amazing meaning. As with all alternatives, the author would appreciate practicality when discussing what could be done. Does the new word actually solve the security and safety issue at hand, is it implementable, and is it acceptable to deploy? Or are we just shooting the shit? The author likes fun, but let's please have the right fun at the right place.

§ 5. Performance

Previous publications [\[Zeroing\]](#) have cited 2.7 to 4.5% averages. This was true because, in the author's experience implementing this in LLVM [\[LLVMJFB\]](#), the compiler simply didn't perform sufficient optimizations. Deploying this change required a variety of optimizations to remove needless work and reduce code size. These optimizations were generally useful for other code, not just automatic variable initialization.

As stated in the introduction, the performance impact is now negligible (less than 0.5% regression) to slightly positive (that is, some code gets *faster* by up to 1%). The code size impact is negligible (smaller than 0.5%).

Why do we sometimes observe a performance progression with zero-initialization? There are a few potential reasons:

- Zeroing a register or stack value can break dependencies, allowing more Instructions Per Cycles (IPC) in an out-of-order CPU by unblocking scheduling of instructions that were waiting on what seemed like a long critical path of instructions but were in fact only false-sharing registers. The register can be renamed to the hardware-internal zero register, and instructions scheduled.

- Zeroing of entire cache entries is magical, though for the stack this is rarely the case. There's questions of temporality, special instructions, and special logic to support zeroed cachelines.

Here are a few of the optimizations which have helped reduce overheads in LLVM:

- CodeGen: use non-zero `memset` for automatic variables [D49771](#)
- Merge clang's `isRepeatedBytePattern` with LLVM's `isBytewiseValue` [D51751](#)
- SelectionDAG: Reuse bigger constants in `memset` [D53181](#)
- ARM64: improve non-zero `memset` isel by ~2x [D51706](#)
- Double sign-extend stores not merged going to stack [D54846](#) and [D54847](#)
- LoopUnroll: support loops w/ exiting headers & uncond latches [D61962](#)
- ConstantMerge: merge common initial sequences [D50593](#)
- IPO should track pointer values which are written to, enabling DSO [DSO](#)
- Missed dead store across basic blocks [pr40527](#)

This doesn't cover all relevant optimizations. Some optimizations will be architecture-specific, meaning that deploying this change to new architectures will require some optimization work to reduce cost.

New codebases using automatic variable initialization might trigger missed-optimizations in compilers and experience a performance or size regression. The wide deployment of the opt-in compiler flag means that this is unlikely, but were this to happen then a bug should be filed against the compiler to fix the missed optimization.

For example, here is Android's experience on performance costs [\[AndroidCosts\]](#).

Some of the performance costs are hidden by modern CPUs being heavily superscalar. Embedded CPUs are often in-order, code running on them might therefore have a performance regression that is roughly proportional to code size increases. Here too, compiler optimizations can significantly reduce these costs.

§ 6. Caveats

There are a few caveats to this proposal.

Making all automatic variables explicitly zero means that developers will come to rely on it. The current status-quo forces developers to express intent, and new code might decide not to do so and simply use the zero that they know to be there. This would then make it impossible to distinguish

"purposeful use of the uninitialized zero" from "accidental use of the uninitialized zero". Tools such as static analysis, memory sanitizers, and valgrind would therefore be unable to diagnose correctness issues (but we would have removed the security issues). It should still be best practice to only assign a value to a variable when this value is meaningful, and only use an "uninitialized" value when meaning has been given to it.

Some types might not have a valid zero value, for example an enum might not assign meaning to zero. Initializing this type to zero is then safe, but not correct. However, C++ has constructors to deal with this type of issue.

For some types, the zero value might be the "dangerous" one, for example a null pointer might be a special sentinel value saying "has all privileges" in a kernel. A concrete example is on Linux where `task->uid` of zero means `root`, uninitialized reads of zero have been the source of CVEs on Linux before.

The core of the above objections is that this change isn't always safe (it's simply safer than uninitialized), and isn't necessarily correct in all circumstances (it's simply correct more often than uninitialized). Sensible people reach different conclusions because of these facts.

This proposal will now mean that objects with dynamic storage duration are now uniquely different from all other storage duration objects (automatic, static, and thread local), in that they alone are uninitialized. We could explore guaranteeing initialization of these objects, but this is a much newer area of research and deployment as shown in [\[XNUHeap\]](#). The strategy for automatically initializing objects of dynamic storage durations depend on a variety of factors, there's no agreed-upon one right solution for all codebases. Each implementation approach has tradeoffs that warrant more study.

Some people think that reading uninitialized stack variables is a good source of randomness to seed a random number generator, but these people are wrong [\[Randomness\]](#).

As an implementation concern for LLVM (and maybe others), not all variables of automatic storage duration are easy to properly zero-initialize. Variables declared in unreachable code, and used later, aren't currently initialized. This includes `goto`, Duff's device, other objectionable uses of `switch`. Such things should rather be a hard-error in any serious codebase.

`volatile` stack variables are weird. That's pre-existing, it's really the language's fault and this proposal keeps it weird. In LLVM, the author opted to initialize all `volatile` stack variables to zero. This is technically acceptable because they don't have any special hardware or external meaning, they're only used to block the optimizer, or in rare cases preserve values across `setjmp` / `longjmp` or signals. The double initialization is technically incorrect because the compiler shouldn't be performing extra accesses to `volatile` variables, but one could argue that the compiler is zero-initializing the

storage right before the `volatile` is constructed. This would be a pointless discussion which the author is happy to have if the counterparty is buying.

§ 7. Wording

Wording for this proposal will be provided once an approach is agreed upon.

The author expects that the solution that is agreed upon maintains the property that:

- Taking the address, passing this address, and manipulating the address of data not explicitly initialized remains valid.
- `memcpy` of data not explicitly initialized (including padding) remains valid, but reading from the data or the `memcpy`d data is currently invalid.

§ 8. Feedback

The first version of this proposal, [\[P2723r0\]](#), has received substantial feedback. This feedback is difficult to summarize, but here are a few (initialized) pointers:

- The author's [original Twitter link](#) generated 100 direct replied (many more sub-replies), received 1.96K likes, 350 retweets, and 304K impressions. Hard to summarize, but most of the interactions were about not having read the paper. Some great feedback came from Twitter, and has been integrated into the paper. The feedback about not having read the paper has been integrated as well, in that the paper hasn't read itself.
- The author's [original Mastodon link](#) generated 40 direct replied (many more sub-replies), received 237 likes, and 128 boosts. The feedback was similar to Twitter, but Mastodon calls it "toots" instead of "tweets" so it's arguably higher quality.
- [lobste.rs](#) had a small discussion, 9 comments in total, the most valuable of which supported the author's dad jokes. Close second stated "HardenedBSD recently switched to auto init to zero by default for the entire userland OS/ecosystem, including 33,000+ packages. Very quite literally zero noticeable performance hit."
- [r/cpp](#) was a thing, with 205 comments. The nuance of these deeps discussions cannot be captured in this paper.
- Slashdot did not discuss, but the author hears that Slashdot is still in existence.
- The [C++ Committee's internal EWG list received 369 emails responding to this paper](#). Some great comments were hidden in these 369 replies, and the author hopes to have captured them in

the first revision of the paper.

- A variety of security professionals have reached out privately to the author, all of their feedback was of the form "ship it".
- Limited feedback from finance / high-frequency trading firms who reached out privately to the author. Their feedback was surprisingly positive "as long as we can opt-out", with a concern about having tooling to help identify potential performance issues.
- Graydon Hoare (creator of Rust, and all-around wonderful human) posted a link to the first version of this paper and [followed up with "\(Speaking of utterly heroic JF Bastien moves\)"](#). The author now questions whether this paper is a conspiracy by the Rust Evangelism Strike Force. 🦀

§ References

§ Informative References

[AndroidCosts]

Alexander Potapenko. [Fighting Uninitialized Memory in the Kernel](#). 2020. URL: https://clangbuiltlinux.github.io/CBL-meetup-2020-slides/glider/Fighting_uninitialized_memory_%40_CBL_Meetup_2020.pdf

[AndroidSec]

Nick Kralevich. [How vulnerabilities help shape security features and mitigations in Android](#). 2016-08-04. URL: <https://www.blackhat.com/docs/us-16/materials/us-16-Kralevich-The-Art-Of-Defense-How-Vulnerabilities-Help-Shape-Security-Features-And-Mitigations-In-Android.pdf>

[AutoInitSummary]

Florian Hahn. [\[llvm-dev\] RFC: Combining Annotation Metadata and Remarks](#). 2020-11-04. URL: <https://lists.llvm.org/pipermail/llvm-dev/2020-November/146393.html>

[CarbonInit]

Chandler Carruth. [Carbon: Initialization of memory and variables](#). 2021-06-22. URL: <https://github.com/carbon-language/carbon-lang/blob/trunk/proposals/p0257.md>

[ChromeUninit]

multiple. [Chromium code search for Use-of-uninitialized-value](#). multiple. URL: <https://bugs.chromium.org/p/chromium/issues/list?can=1&q=Stability%3DMemory-MemorySanitizer+-status%3ADuplicate+-status%3AWontFix+Use-of-uninitialized-value>

[CLessDangerous]

Kees Cook. [Making C Less Dangerous](#). 2018-08-28. URL: <https://outflux.net/slides/2018/lss/danger.pdf>

[CoreGuidelines]

Herb Sutter; Bjarne Stroustrup. C++ Core Guidelines: ES.20: Always initialize an object. unknown. URL: <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Res-always>

[CppConComplexity]

Herb Sutter. Empirically Measuring, & Reducing, C++'s Accidental Complexity. 2020-10-11. URL: <https://www.youtube.com/watch?v=6lurOCdaj0Y&t=971s>

[CppConDefinite]

Herb Sutter. Can C++ be 10x Simpler & Safer?. 2022-11-13. URL: <https://www.youtube.com/clip/UgkxpWxXNaoWvGqhlpSdWRQs0M3ayQcEhDoh>

[CppConMSRC]

Joseph Bialek; Shayne Hiet-Block. Killing Uninitialized Memory: Protecting the OS Without Destroying Performance. 2019-10-13. URL: <https://www.youtube.com/watch?v=rQWjF8NvqAU>

[CWE-457]

MITRE. CWE-457: Use of Uninitialized Variable. 2006-07-19. URL: <https://cwe.mitre.org/data/definitions/457.html>

[D2488R0]

Andrzej Krzemiński. Attribute `[[for_overwrite]]`. 2021-11-11. URL: <https://isocpp.org/files/papers/D2488R0.html>

[DR451]

Freek Wiedijk; Robbert Krebbers. Instability of uninitialized automatic variables. 2013-08-30. URL: https://www.open-std.org/jtc1/sc22/wg14/www/docs/dr_451.htm

[GCCReview]

Qing Zhao. [RFC][patch for gcc12][version 1] add -ftrivial-auto-var-init and variable attribute "uninitialized" to gcc. 2021-02-18. URL: <https://gcc.gnu.org/pipermail/gcc-patches/2021-February/565514.html>

[GoogleNumbers]

Kostya Serebryany. [cfe-dev] [RFC] automatic variable initialization. 2019-01-16. URL: <https://lists.lvm.org/pipermail/cfe-dev/2019-January/060878.html>

[INIT-LOCAL-VARS]

Florian Weimer. -finit-local-vars option. 2014-06-06. URL: <https://gcc.gnu.org/ml/gcc-patches/2014-06/msg00615.html>

[InitAll]

Joseph Bialek. Solving Uninitialized Stack Memory on Windows. 2020-05-13. URL: <https://msrc-blog.microsoft.com/2020/05/13/solving-uninitialized-stack-memory-on-windows/>

[KCC]

Kostya Serebryany. [cfe-dev] [RFC] automatic variable initialization. 2018-11-15. URL: <https://lists.lvm.org/pipermail/cfe-dev/2018-November/060177.html>

[Linus]

Linus Torvalds. [Re: \[GIT PULL\] meminit fix for v5.3-rc2](#). 2019-06-30. URL: https://lore.kernel.org/lkml/CAHk-wgTM+cN7zyUZacGQDv3Duu0A4LORNPWgb1Y_Z1p4iedNQ@mail.gmail.com/

[LinuxExploits]

Kees Cook. [Kernel Exploitation via Uninitialized Stack](#). 2019-08. URL: <https://outflux.net/slides/2011/defcon/kernel-exploitation.pdf>

[LinuxInfoleak]

Mathias Krause. [Linux kernel: net - three info leaks in rtnl](#). 2013-03-19. URL: <https://seclists.org/oss-sec/2013/q1/693>

[LinuxUse]

Kangjie Lu; et al. [Unleashing Use-Before-Initialization Vulnerabilities in the Linux Kernel Using Targeted Stack Spraying](#). 2017-03-01. URL: <https://www-users.cs.umn.edu/~kjl/papers/tss.pdf>

[LLVMJFB]

JF Bastien. [Mitigating Undefined Behavior](#). 2019-12-12. URL: <https://www.youtube.com/watch?v=I-XUHPimq3o>

[LLVMReview]

JF Bastien. [Automatic variable initialization](#). 2018-11-15. URL: <https://reviews.llvm.org/D54604>

[LocalInit]

thesting. [add LocalInit sanitizer](#). 2018-06-22. URL: https://github.com/AndroidHardeningArchive/platform_external_clang/commit/776a0955ef6686d23a82d2e6a3cbd4a6a882c31c

[N1793]

Freek Wiedijk; Robbert Krebbers. [Stability of indeterminate values in C11](#). URL: <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n1793.pdf>

[NaT]

Raymond Chen. [Uninitialized garbage on ia64 can be deadly](#). 2004-01-19. URL: <https://devblogs.microsoft.com/oldnewthing/20040119-00/?p=41003>

[P0146r1]

Matt Calabrese. [Regular Void](#). 11 February 2016. URL: <https://wg21.link/p0146r1>

[P0632r0]

Jonathan Müller. [Proposal of \[\[uninitialized\]\] attribute](#). 19 January 2017. URL: <https://wg21.link/p0632r0>

[P2687r0]

Bjarne Stroustrup, Gabriel Dos Reis. [Design Alternatives for Type-and-Resource Safe C++](#). 15 October 2022. URL: <https://wg21.link/p2687r0>

[P2723r0]

JF Bastien. [Zero-initialize objects of automatic storage duration](https://wg21.link/p2723r0). 16 November 2022. URL: <https://wg21.link/p2723r0>

[Randomness]

Xi Wang. [More randomness or less](https://kqueue.org/blog/2012/06/25/more-randomness-or-less/). 2012-06-25. URL: <https://kqueue.org/blog/2012/06/25/more-randomness-or-less/>

[Seacord]

Robert C. Seacord. [Uninitialized Reads: Understanding the proposed revisions to the C language](https://queue.acm.org/detail.cfm?id=3041020). 2017-01-16. URL: <https://queue.acm.org/detail.cfm?id=3041020>

[Trap]

Richard Smith. [\[NOT FOR REVIEW\] Experimental support for zero-or-trap behavior for uninitialized variables](https://reviews.llvm.org/D79249). 2020-05-01. URL: <https://reviews.llvm.org/D79249>

[WinKernel]

Joseph Bialek. [Join the Windows kernel in wishing farewell to uninitialized plain-old-data structs on the stack](https://twitter.com/JosephBialek/status/1062774315098112001). 2018-11-14. URL: <https://twitter.com/JosephBialek/status/1062774315098112001>

[XNUHeap]

Apple Security Engineering and Architecture. [Towards the next generation of XNU memory safety: kalloc_type](https://security.apple.com/blog/towards-the-next-generation-of-xnu-memory-safety/). 2022-10-22. URL: <https://security.apple.com/blog/towards-the-next-generation-of-xnu-memory-safety/>

[Zeroing]

Xi Yang; et al. [Why Nothing Matters: The Impact of Zeroing](https://users.elis.ugent.be/~jsartor/researchDocs/OOPSLA2011Zero-submit.pdf). 2011-10. URL: <https://users.elis.ugent.be/~jsartor/researchDocs/OOPSLA2011Zero-submit.pdf>